IITM PRAVARTAK
CATALYSING INNOVATION

# Professional Certificate Programme in Agentic AI and Applications

# Agentic Tools in Python

# Revisit the Learning So Far

Let’s take a quick look at the key ideas and concepts you have explored in your learning journey.

- How embeddings capture meaning by converting text into numerical vectors
- Distinction between token vectors and sentence embeddings and when each is applied
- How cosine and Euclidean distance measure similarity between vectors
- How nearest neighbours are identified in vector space to find related text
- How embeddings can be visualised using dimensionality reduction techniques
- How vector search uses embeddings to retrieve relevant information
- A journaling activity to reflect on key insights about embeddings

# Topics Covered

- Introduction to LangChain
- Prompt Formatting & Response Parsing
- Prompt Formatting
- Response Parsing
- LangChian Tools & Tool Calling
- Introduction to Prompt Templates

# Introduction to LangChain

Building applications with Large Language Models (LLMs) that are practical, powerful and production-ready for real-world deployment.

# Why LangChain

## LLMs are Powerful but Limited


**Context window constraints**

Struggle with long contexts and lose information beyond token limits

**No memory persistence**

Cannot retain details from past interactions or conversations.

**Data access limitations**

No direct access to external databases, APIs or real-time data

# Why LangChain?

**LangChain solves this by:**

- Adding persistent memory
- Connecting to data sources
- Orchestrating multiple components

# Core Idea of LangChain

**LangChain is a framework that connects isolated LLMs into intelligent systems for real-world use.**

| Connect LLMs with external tools | Manage chains of reasoning | Store and retrieve knowledge | Enable real-world AI applications |
| --- | --- | --- | --- |
| Integrate APIs, databases, web search and custom functions. | Handle sequential processes and complex workflows | Support persistent memory and document search | Build production-ready, scalable AI systems. |

# Main Components of LangChain

**LLMs**

Core language models from OpenAI, Hugging Face, Anthropic and others

**Prompts**

Structured templates to optimise and standardise model inputs

**Chains**

Sequential workflows combining models, tools and processes

**Agents**

Autonomous systems that select actions and tools based on context

**Memory**

Persistent storage of conversation history and context

**VectorStores**

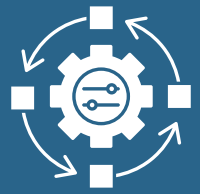
Efficient systems for semantic search and document retrieval

# How it Works

## Basic Flow

**User input**
Natural language query or command


**Prompt template**
Structure and format the input

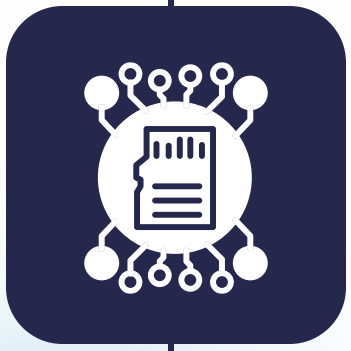


**LLM processing**
Generate intelligent response

**Output**
Final result to user

# How it Works

## Enhanced with LangChain



**Memory integration**
Conversation-aware applications that remember context

**Document retrieval**
Search company documents, PDFs and databases

# Example Use Cases


### Intelligent chatbots

Provide context-aware customer service, remembering conversation history and user preferences.

### Document Q&A systems

Answer questions using company documents, legal contracts and knowledge bases.

### Code assistants

Support development by connecting to APIs, documentation and version control

# Example Use Cases

**Research assistants**

Retrieve and synthesise information from scientific papers

**RAG applications**

Combine real-time data retrieval with AI-generated content

# Simple Demo (Conceptual)

```
from langchain import OpenAI, ConversationChain

# Initialize the LLM
llm = OpenAI(temperature=0)

# Create a conversation chain
conversation = ConversationChain(llm=llm)

# Interactions
response1 = conversation.run("Hello, who won the 2018 FIFA World Cup?")
print(response1)  # Expected: "France won the 2018 FIFA World Cup"

response2 = conversation.run("What was the final score?")
print(response2)  # LangChain remembers context and responds appropriately
```

# Simple Demo (Conceptual)

## Key Difference

| Without Langchain | With LangChain |
| --- | --- |
| LLM answers once, no context retention | App maintains conversation memory and context awareness |

# Architecture Overview

**Input layer**
Receive user prompts, queries and natural language commands

**Core processing**
Orchestrates workflows with Chains and enable dynamic decisions via Agents

**Memory layer**
Stores conversation history and contextual state persistently

**Data layer**
Uses vector databases like Pinecone, FAISS and Chroma for semantic search

**Output layer**
Delivers structured responses, actions and final results to users.

# Ecosystem & Integrations

**Vector databases**
FAISS, Pinecone, Weaviate, Chroma for similarity search

**LLM providers**
OpenAI, Hugging Face, Anthropic, Cohere, local models


**Monitoring**
LangSmith for debugging and performance optimisation

**External tools**
Google Search, APIs, SQLdatabases and web scraping tools

# Benefits

**Rapid prototyping**
Builds AI applications swiftly using pre-built components and detailed documentation

**Modular Flexibility**
Plug-and-play architecture allows easy swapping of models, tools, and data sources

**Scalable architecture**
Supports both small experiments and large-scale production deployments seamlessly

**Rich ecosystem**
Offers extensive integrations with popular tools, databases and AI services

# Limitations & Challenges

Adds unnecessary complexity for simple use cases without advanced needs

Depends on external vector databases, APIs and third-party services

Requires frequent updates due to fast-changing library and maintenance

**Complexity overhead**

**External dependencies**

**Rapid evolution**

**Consider the trade-offs:** Assess if LangChain's full framework is necessary or if simpler solutions suffice.

# Demo: Set up LangChain and OpenAI integration, execute a simple LLMChain

# Prompt Formatting & Response Parsing

Designing inputs and structuring outputs for reliable LLM use

# Building Reliable AI Systems


**Challenge**
LLMs produce varied natural language responses, complicating workflows needing consistent, structured outputs

**Solution**
Apply strategic prompt formatting and robust parsing to ensure reliable, deployable AI applications

# Why Structure Matters


**Natural language output**

LLMs generate varied, human-like responses that differ in format, length and structure each time.

**Production requirements**

Applications need predictable, parseable data formats for APIs, databases and user interfaces.

# Why Structure Matters

**Prompt formatting**

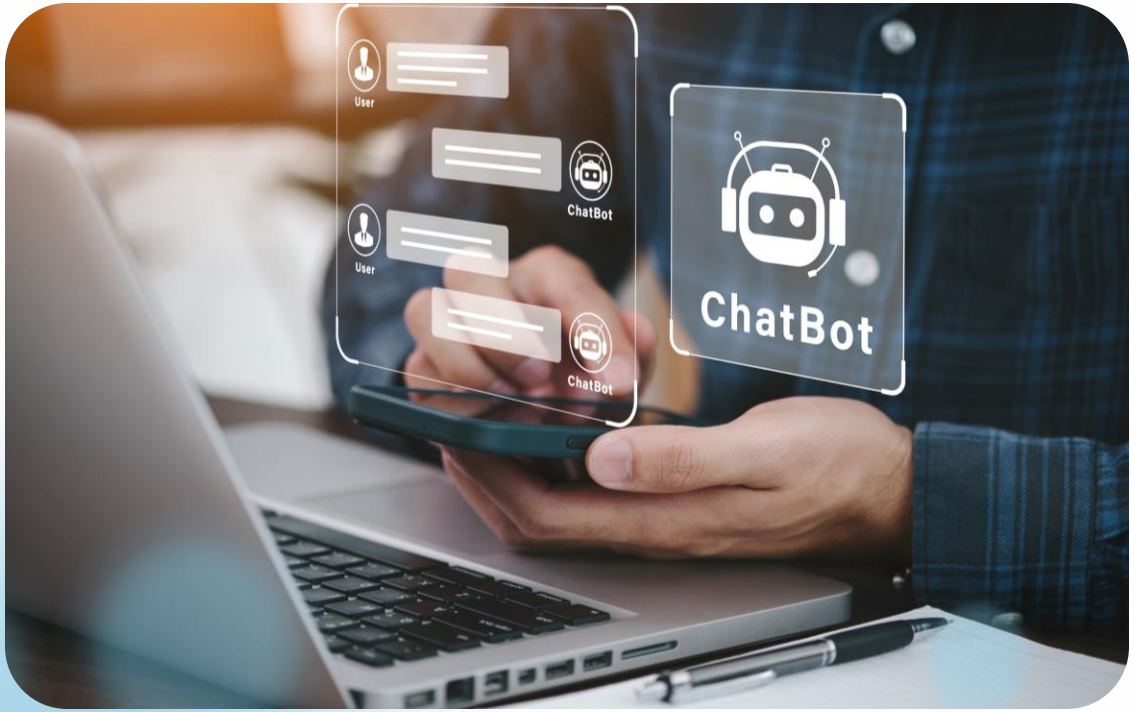
Provide clear, structured instructions to guide model behaviour and output format.

**Response parsing**

Extract usable data in formats like JSON, tables, or structured text from model outputs.

# Prompt Formatting

# What is Prompt Formatting




**Strategic input design**
Structure instructions, context and constraints to guide LLM behaviour

**Clarity through structure**
Set clear expectations and formats to reduce ambiguity and increase predictability

**Human communication principles**
Ask precise, well-structured questions to elicit actionable responses

# Building Blocks of Effective Prompts

**Context**
Provide background, domain knowledge and situational awareness to frame the task.

**Instruction**
Give clear, specific directions including constraints and requirements.

**Format specification**
Define explicit guidelines for output structure, format and organisation.

**Examples**
Offer few-shot samples demonstrating the desired style, format and detail.

# Prompt Example – Weak vs Strong

**Poor prompt**

"Answer the employee’s question about HR policies."

- Instructions lack clarity
- No output format specified
- Output length varies
- Hard to parse programmatically

# Prompt Example – Weak vs Strong

**Better prompt**

"You are an internal HR Policy Assistant.
Use ONLY the policy documents provided in the CONTEXT section below.
For the given QUESTION, do the following:

1. Provide a short, clear answer in 2–4 sentences.
2. Cite the specific policy section(s) or clause IDs you used.
3. If the policy does not explicitly cover the question, say so clearly instead of guessing.
4. Do not invent policies or benefits that are not stated in the context.

CONTEXT:
{retrieved_policy_chunks}
QUESTION:
{employee_question}"

- Defined output format
- Fixed length guideline
- Organised structure
- Easily parseable results

# Prompt Engineering Techniques

**Zero-shot prompting**

Give direct instructions without examples for simple, well-defined tasks

Provide 2–5 input-output examples to guide format and style

**Few-shot prompting**

**Chain-of-thought prompting**

Request step-by-step reasoning to enhance accuracy in complex tasks

Specify exact output formats using JSON schemas, CSV templates or Markdown structures for consistent parsing

**Schema-guided prompting**

# Prompt Formatting Best Practices


**Be specific**
Define what you want, how you want it and any constraints to ensure consistency.

**Set clear boundaries**
Specify output length, format, and structure to simplify parsing and reduce complexity.

**Use consistent language**
Maintain consistent terms and phrasing to improve reliability and clarity

**Test & iterate**
Refine prompts continuously based on model outputs to optimise results

# Response Parsing

# What is Response Parsing

## Extracting Structure from Chaos

Transforms LLM's free-form natural language output into structured, machine-readable data formats

Enables seamless integration with APIs, databases, analytics platforms and user interfaces

Supports the shift from experimental prototypes to production-ready applications

# Response Parsing Approaches

**Simple text cleaning**
Use regex and string matching to extract information from unstructured responses

**Structured output requests**
Specify desired formats such as JSON, YAML or Markdown tables in the prompt

**Parser libraries**
Employ frameworks like LangChain's OutputParser, Guardrails AI or Pydantic for validation

**Post-processing logic**
Conduct validation, normalisation and error handling to ensure data integrity

# JSON Output Parsing Example

**Structured prompt:** Extract company name and revenue from the text below. Respond in JSON format only with keys 'company' and 'revenue

**Clean response:**

```
{  "company": "OpenAI",  "revenue": "$1B+"}
```

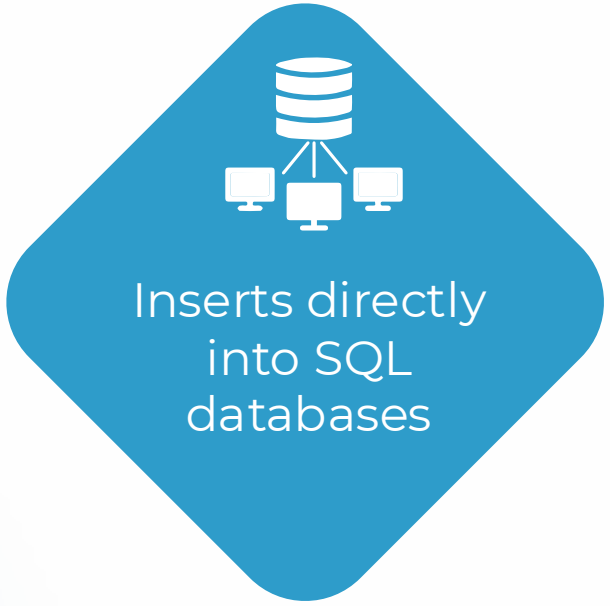
# Structured Output for Downstream Use

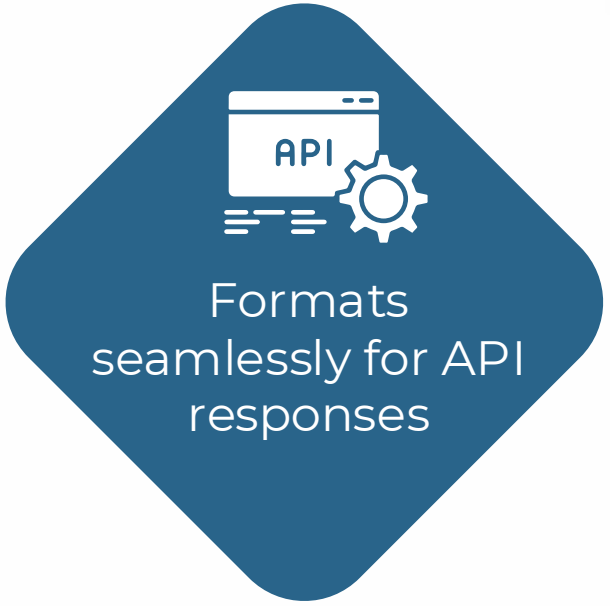
**Ask the model to respond in strict JSON:**

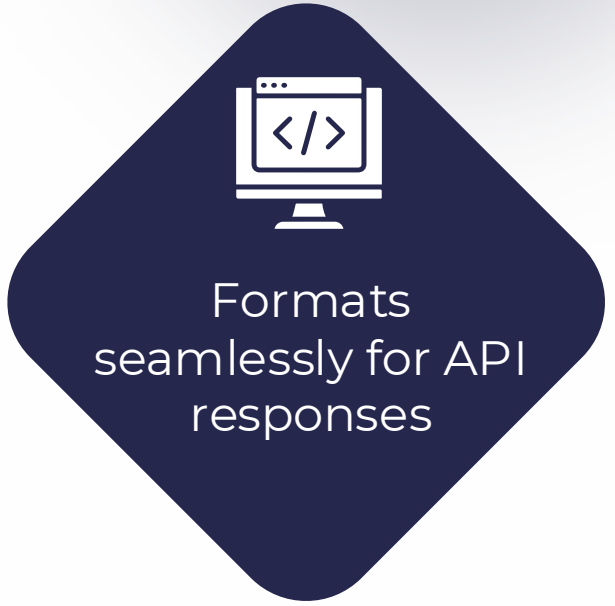
```json
{
  "short_answer": "<2-4 sentence answer>",
  "policy_citations": [
    {"document": "Leave_Policy.pdf", "section": "3.2"},
    {"document": "Contract_Benefits.pdf", "section": "1.4"}
  ],
  "needs_escalation": true,
  "escalation_reason": "Policy does not cover contract staff in probation period."
}
```

# JSON Output Parsing Example

## Integration Benefits

# Common Parsing Challenges

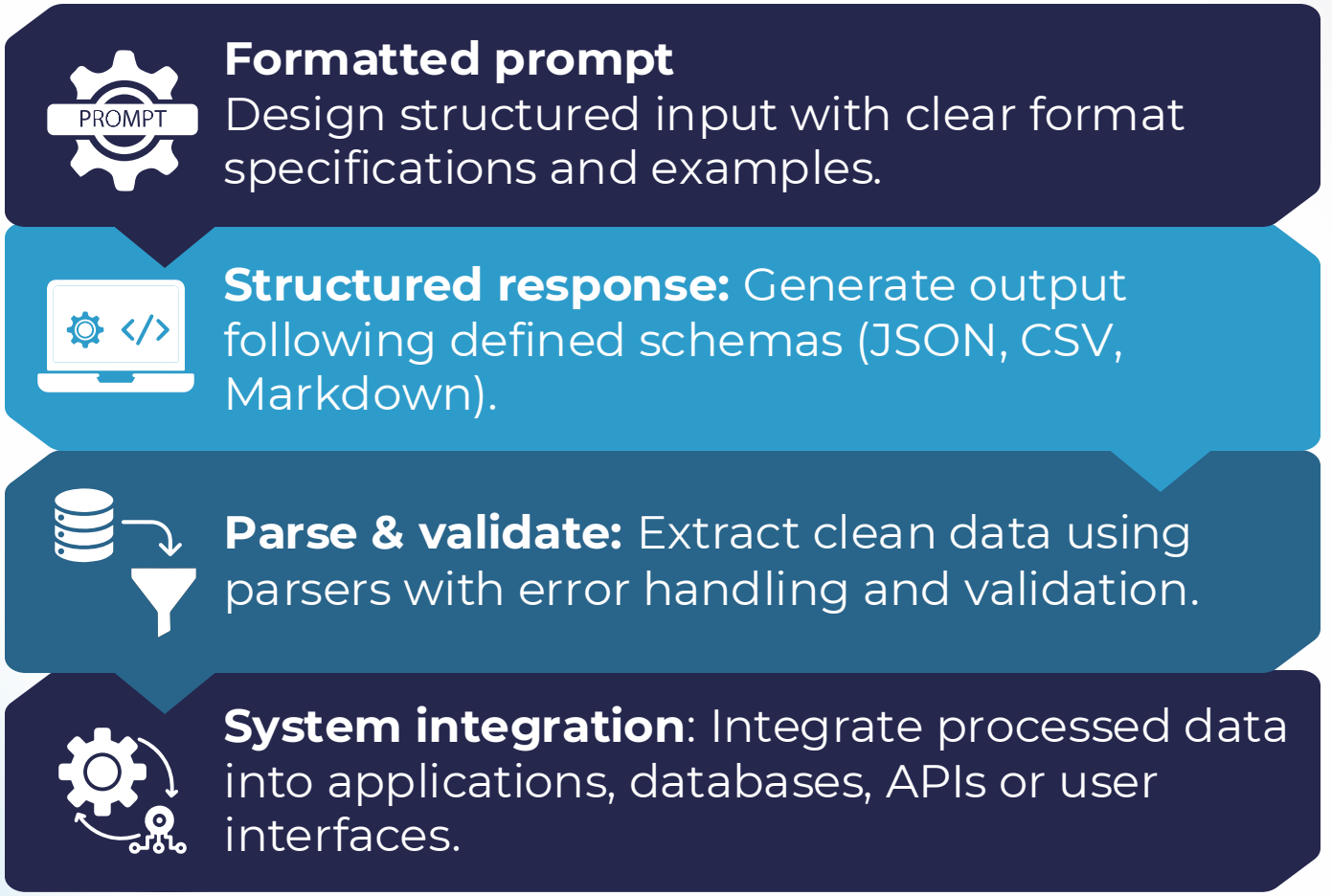
**Format inconsistency**
Models may ignore format rules or add unexpected text.

**Incomplete structures**
Missing fields, malformed JSON or inconsistent data disrupt parsing.

**Validation failures**
Outputs may not match schemas or contain invalid values.

**Robust solutions**
Use schema enforcement, retries, fallback parsing and thorough validation.

# Production Workflow



# Real-World Applications

### Business Intelligence

Automated report summarisation with KPI extraction for executive dashboards and performance monitoring.

### Conversational APIs

Structured chatbot responses integrated with business systems and customer service workflows.

# Real-World Applications

### Data processing

Automated labelling, classification and entity extraction for large data pipelines.


### RAG systems

Controlled formatting of retrieved answers in Retrieval-Augmented Generation with consistent structure.


Master these techniques to build reliable, production-ready AI systems that integrate seamlessly with your infrastructure.

# LangChain Tools & Tool Calling

Enhancing LLMs with External Capabilities

# The Challenge: LLM Limitations

Large Language Models are powerful but face key constraints limiting real-world use.

**Mathematical reliability**: Struggle with precise calculations and complex arithmetic operations

**Knowledge cutoffs**: Use outdated training data lacking current events and real-time information

**Structured outputs**: Finds difficulty in producing consistent, API-compatible responses

**Solution:** Extend LLMs with external tools - functions, APIs and utilities that bridge these capability gaps.

# What are LangChain Tools

Tools in LangChain are predefined or custom functions that an LLM calls to extend capabilities beyond text generation.

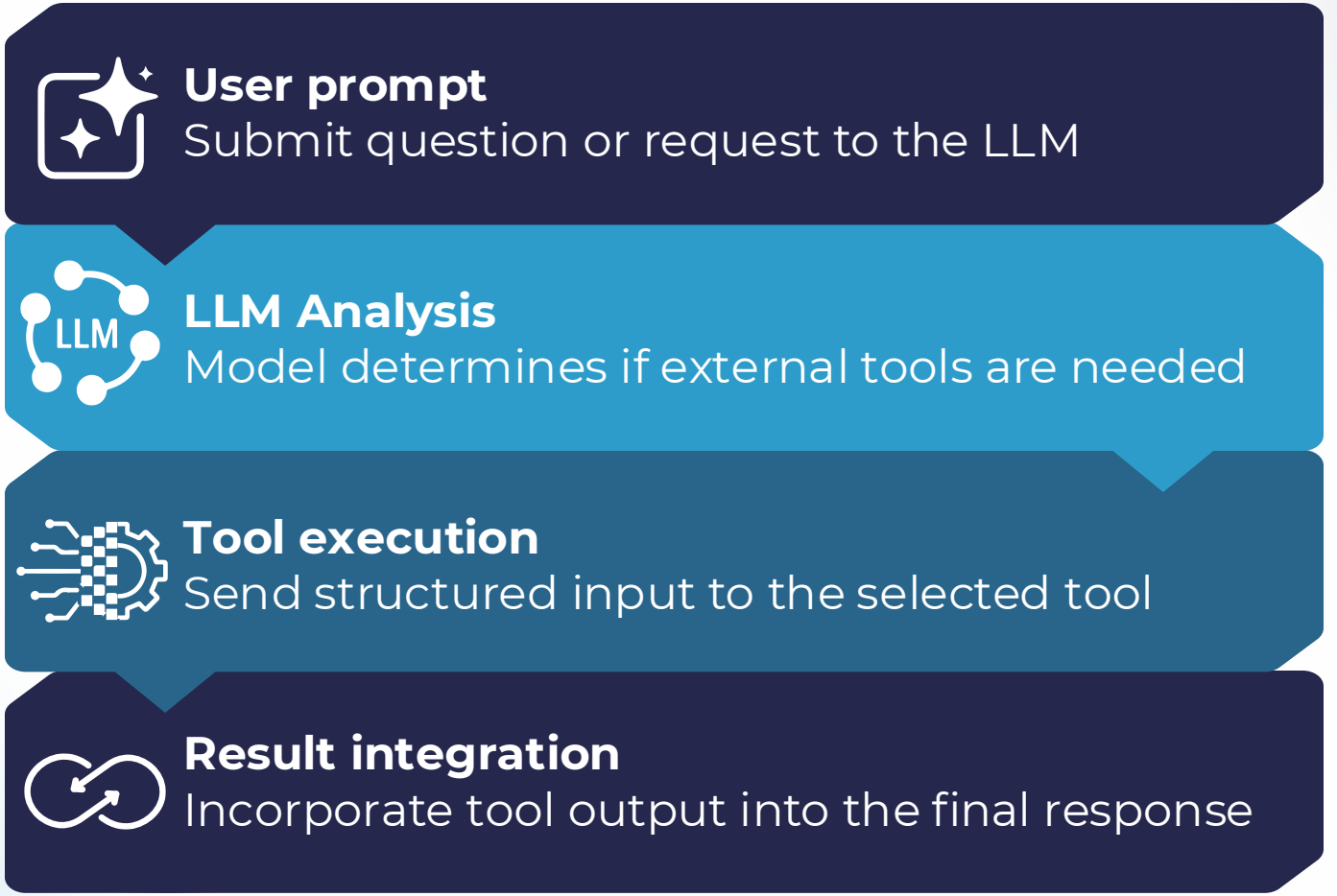
**Mathematical operations**
Precision calculators for arithmetic, algebra and complex computations

**Information retrieval**
Search engines and web scrapers for real-time, up-to-date data access

**Data integration**
Direct connections to databases, APIs, and enterprise systems

**Custom logic**
Business-specific functions and workflows tailored to your needs

# Tool Calling Flow Architecture




This workflow lets LLMs autonomously decide when and how to use external capabilities.

# Example: Mathematical Precision

**User prompt:**
"What is (345 × 23) + 99?"


**Pattern recognition**
LLM detects a mathematical expression needing calculation

**Tool invocation**
Call calculator tool with structured input: "(345*23)+99“

**Precise computation**
Tool returns the accurate result: 8,034

**Natural response**
LLM formats the answer: "The result is 8,034"

Tools eliminate calculation errors and provide reliable mathematical results every time.

# Example: Real-Time Information

**User query:**
"Who is the current CEO of Google?"


**Query analysis**
LLM recognises need for current information beyond training data

**Search tool activation**
Web search tool queries for "current Google CEO“

**Data retrieval**
Tool returns: "Sundar Pichai“

**Contextual response:**
"The current CEO of Google is Sundar Pichai"

# Code Implementation Setup

## Initialising LangChain with Multiple Tools for Enhanced LLM Capabilities

```python
# Import necessary modules
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, load_tools

# Initialize the LLM with deterministic behavior
llm = ChatOpenAI(temperature=0)

# Load pre-built tools for search and mathematics
tools = load_tools(["serpapi", "llm-math"], llm=llm)

# Create an agent that can reason about tool usage
agent = initialize_agent( tools=tools, llm=llm,  agent="zero-shot-react-description")

# Execute a complex query requiring multiple tools
result = agent.run(
    "What is the square root of 256 plus the CEO of Google's first name?")

# Display the result
print(result)
```

This setup lets the LLM autonomously select and use tools.

# Observing Multi-Tool Execution

## Demonstrating Sophisticated Reasoning in Processing Complex Queries

Recognises "square root of 256" requires calculator tool

Result: $\sqrt{256} = 16$

**Mathematical analysis**

Identifies need for current CEO information

Result: "Sundar" (first name)

**Information lookup**

Combines numerical and text results

Final: "16 + Sundar"

**Intelligent synthesis**

This shows the agent’s ability to coordinate multiple tools within one workflow.

# Why Tool Calling Transforms AI


**Action-oriented intelligence:** Turns LLMs from text generators into problem solvers

**Reduced hallucinations:** Bases responses on verified external data

**Real-world integration:** Links AI systems to live data and operations

**Domain expertise:** Supports industry-specific, tailored workflows

Tool calling connects AI capabilities with real-world business applications.

# Real-World Applications

## Powerful, Domain-Specific AI Applications Enabled by Tool Calling

### Enterprise integration

Direct queries to CRM, ERP and business intelligence systems for real-time insights and automated reports.


### Educational technology

Intelligent tutoring systems offering step-by-step explanations with accurate calculations and verification.

# Real-World Applications

## Powerful, Domain-Specific AI Applications Enabled by Tool Calling

### Research & analytics

Automated data retrieval, analysis and insights from live data sources and scientific databases.

### Process automation

On-demand execution of scripts, workflows and business processes via natural language commands.

# Demo

# Introduction to Prompt Templates

Structuring prompts for reusability and building intelligent agents

# Building on What We Know

## Well-Formatted Prompts Improve LLM Responses, but Production Presents New Challenges

Reuse prompts across different tasks and contexts

Insert dynamic variables such as {question} and {context}

Ensure consistency while supporting flexibility

Prompt Templates provide the structured solution we need.

# What is a Prompt Template

Use a structured prompt framework with placeholder variables filled dynamically at runtime

**Reusable pattern**

Replace variables such as {context} and {question} with actual values during execution

**Dynamic content**

Ensures standardised instructions while adapting to varied inputs and use cases

**Consistency + Flexibility**

**Example prompt:**
"Answer the following question based on the provided context:\n Context: {context}\n Question: {question}\n Answer:"

# Why Prompt Templates Matters

**Consistency:** Standardise instructions and formatting across tasks to reduce variability and enhance reliability.

**Reusability:** Use one template to handle thousands of varied queries.

**Control:** Enforce specific response structures, tone and style while allowing flexibility for different contexts.

**Integration:** Pass dynamic variables from applications to LLMs for real-time, context-aware responses.

# Practical Example: Q&A Template

```python
# Import the PromptTemplate class
from langchain.prompts import PromptTemplate
# Step 1: Define the template
template = """Answer the question based on the context below:

Context: {context}
Question: {question}
Answer:"""

# Step 2: Create the PromptTemplate
prompt = PromptTemplate(input_variables=["context", "question"], template=template)

# Step 3: Format the template with actual values
print(
    prompt.format(
        context="Paris is the capital of France.",
        question="What is the capital of France?" )
```

**Output:**

"Answer the question based on the context below: Context: Paris is the capital of France. Question: What is the capital of France? Answer: Paris"

# Templates Power AI Agents

## What are AI Agents?

Intelligent systems that combine LLMs with reasoning capabilities and external tools.


**They can:**

- Analyse problems step-by-step
- Choose appropriate tools
- Execute actions in sequence
- Synthesise final answers

Prompt templates guide their reasoning process, ensuring consistent decision-making patterns.

# Templates Power AI Agents

## Example


An agent uses templates to structure problem-solving, tool usage and result presentation

# Building Your First Agent

**Define available tools:** Set up calculator and search functions your agent can use to solve problems.

**Create reasoning template:** Design a prompt guiding the model to think through problems systematically.

**Initialise agent framework**: Combine the LLM, tools and template into a cohesive, reasoning agent.

**Execute and observe**: Run queries and watch the agent analyse problems, utilise tools and produce answers.

# Agent Implementation Code

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, load_tools
from langchain.prompts import PromptTemplate

llm = ChatOpenAI(temperature=0)
tools = load_tools(["serpapi", "llm-math"], llm=llm)

prompt = PromptTemplate(input_variables=["question"], template="You are a smart assistant
    . Use tools when needed.\nAnswer clearly and show your reasoning.\n\nQuestion:
    {question}\nAnswer:")

agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)

print(agent.run(prompt.format(question="What is the square root of 144 plus the current
    US President's first name?")))
```

# Agent Reasoning Flow

## Template processing

LLM reads the structured template: "You are a smart assistant. Use tools when needed...“

## Tool selection & use

Agent identifies the need for calculator, computes √(144=12), then searches for current US President

## Result synthesis

Combines mathematical result with search data to produce final structured answer

# Reflect

## Where Could I Use Agentic Tools in My Own Role/Workflow?

Identify tasks like summarising reports, drafting emails and cleaning data.

Spot decision-making steps for quick insights, such as market trends and financial projections.

Consider time-consuming research that agents could accelerate, like gathering industry benchmarks.

Explore workflows combining tools, for example, search and calculator for business analysis.

Ask, “*If I had a digital co-worker with access to search, memory, and reasoning, what tasks would I hand off?*”

# Key Takeaways

- **Templates enable scale:** Prompt templates provide structure and reusability for consistent, production-ready AI systems.
- **Agents extend capabilities:** Combining LLMs, reasoning templates, and tools enables autonomous solving of complex, multi-step problems.
- **Foundation for innovation:** Simple agents highlight the synergy of prompts, reasoning and tools in AI workflows

Ready to build intelligent, scalable AI systems?

# Q & A

# Thank you